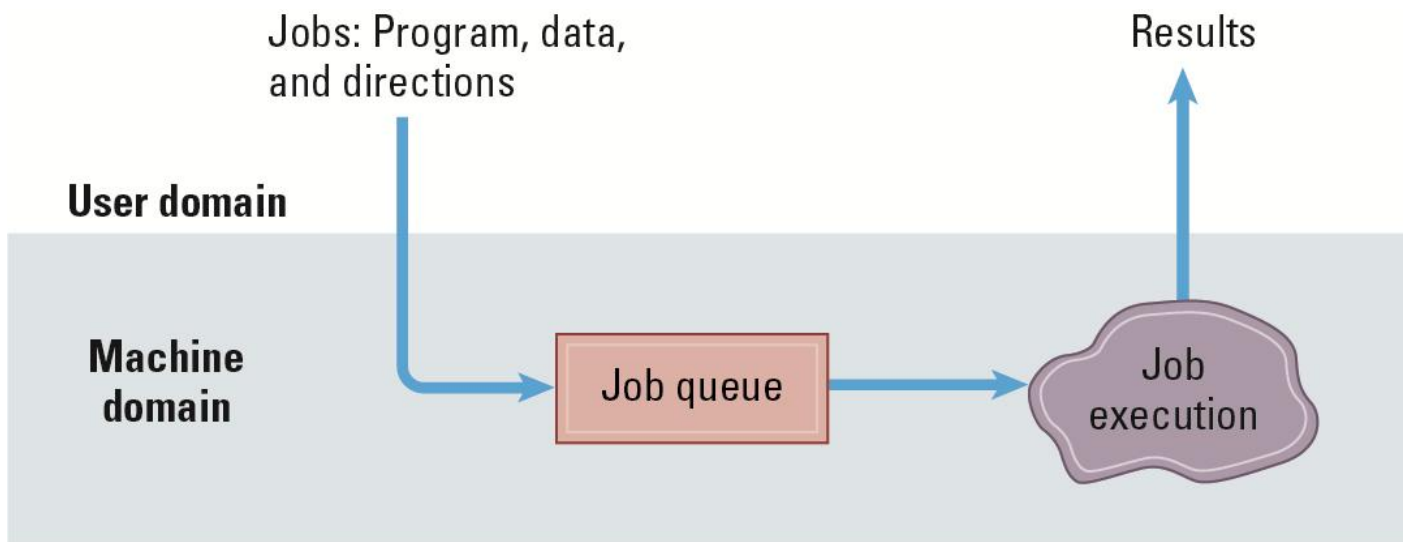
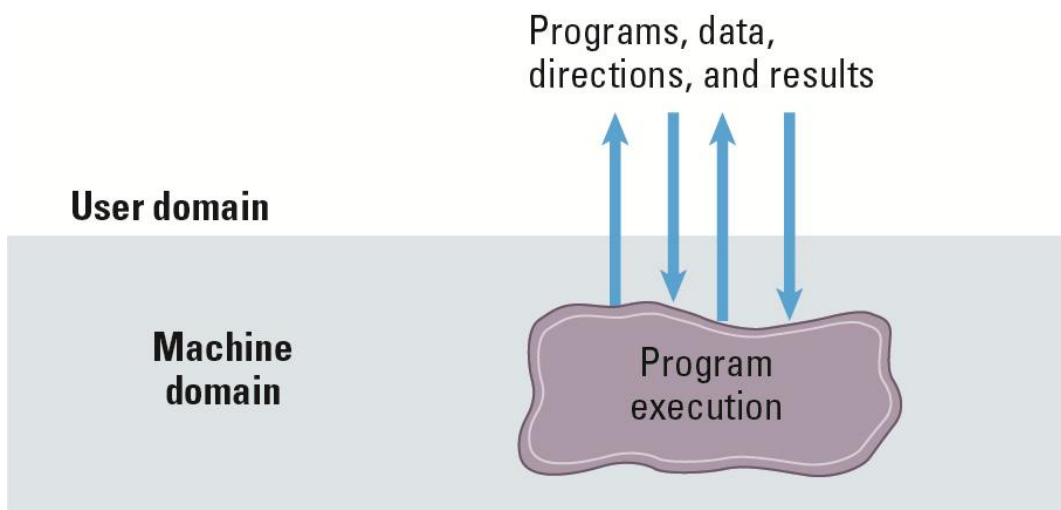


作業系統

Batch Processing : job queue · FIFO



Interactive Processing : real time



Time-sharing (一機多人)

透過 multiprogramming 將時間分割，快速切換執行的程式，讓使用者感覺同時在運算

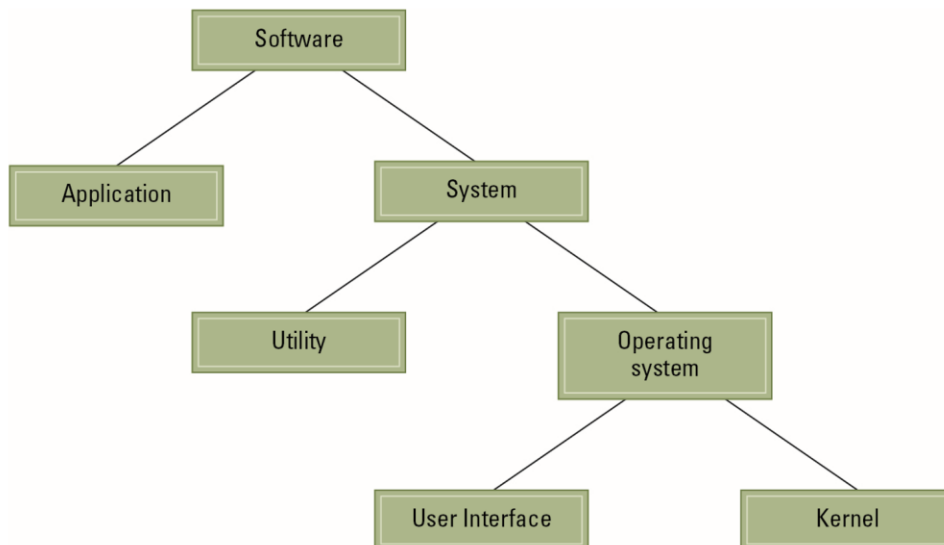
- Time slice：系統中執行的最小時間片段，通常在 10 毫秒~10 微秒
- Dispatcher (派遣器)：根據 Process Table 切換執行的行程
- Process Switch=Context Switch：切換正在執行的 process 的過程，由 Dispatcher 完成

Multitasking (單人多工)

Multiprocessor：一臺電腦有多 CPU 提升效能

- Load balancing：動態調整工作分配到各處理器
- Scaling：將大 Task 依照目前可用的處理器分割成 Subtask，避免有 CPU 空間或過載

Software classification



- OS
 - UI
 - Shell
 - GUI
 - Kernel
 - Scheduler
 - Dispatcher
 - File Manager
 - Directory=Folder 資料夾
 - Path : `/path/to/directory`
 - Memory Manager
 - Main Memory
 - Virtual Memory (Paging)

Memory Paging

◆ 記憶體不足與「分頁 (Paging) 」技術

當主記憶體實際容量不足以容納所有需要的程式與資料時，
記憶體管理器會使用一種技巧來製造「更多記憶體」的假象——
這個技巧叫作 分頁 (paging) 。

■ 例子：

假設一個系統執行的任務需要 8GB 的主記憶體，
但電腦實際上只有 4GB。
記憶體管理器會：

1. 在**磁碟 (mass storage) **上保留 4GB 空間；
2. 在那裡儲存那些「如果主記憶體有 8GB」時才會存在的資料；
3. 將整體資料分成許多固定大小的單位，稱為 頁面 (pages)，通常每頁是數 KB；
4. 不斷地在主記憶體與磁碟之間交換 (shuffle) 頁面，
確保目前正在使用的頁面會被保留在主記憶體中。

如此一來，系統就能假裝自己擁有 8GB 記憶體，
雖然實際上只有 4GB。

這個由分頁技術所創造的「虛擬」記憶體空間，稱為 虛擬記憶體 (virtual memory) 。

Bootling：電腦通電到載入 OS 的過程

- Bootloader：存在 ROM/Flash 非揮發性記憶體裡面，讓硬體初始化的程式
- 現代電腦 Main Memory 也有留一小塊 **nonvolatile memory cell** 儲存 Bootloader
但大多還是在 Flash
- CPU 的電路設計會在通電後執行 Bootloader 的程式
- 按下電源鍵
→CPU 通電執行 Bootloader

- Bootloader 初始化硬體
- Bootloader 載入 OS 到 Main Memory
- Bootloader 指揮 CPU JUMP 到已載入好的 OS
- OS 接管控制權

*不用初始化 CPU，電路設計上會直接執行 bootloader

Bootloader 儲存在 ROM 或 Flash，

嚴格來說仍是可以更新的 (Firmware Update 韌體更新)

Process 行程

- Process State：這個實體的狀態
 - Program Counter
 - General purpose registers
 - Related portion of memory
- 由 Scheduler 和 Dispatcher 管理

Scheduler

- Process Table：系統用來管理 Process 的資料結構
- Kernel 負責增減 Process Table
- Scheduler：根據 Process Table 決定下一個要執行的行程，**排程決策**
- Dispatcher：把 Scheduler 做的排程決策實際切換過去

Scheduler：問誰上場？

Dispatcher：推他上場！

Critical Region

一段會存取共享資源的程式碼區塊，若多個程序或執行緒同時執行這段程式，可能導致競爭（race condition）而資料不一致

💡 二、範例

假設有兩個執行緒同時修改同一個變數：

```
c
// 共享變數
int counter = 0;

void increment() {
    counter = counter + 1;
}
```

若兩個執行緒同時執行 `increment()`：

- Thread A 讀到 counter=0
- Thread B 也讀到 counter=0
- A、B 都加一，然後寫回 → 結果變成 1（而不是 2）

這段修改 `counter` 的程式區塊就是 **critical region**。

想像：

- 「共享資源」= 洗手間 🚻
- 「臨界區」= 進去洗手間那段時間
- 「lock/unlock」= 關門上鎖與打開門

你不希望兩個人同時進去同一間廁所 😊
→ 這就是 **Critical Region** 保護的意義。

Semaphore 旗標

- 旗標表示共享資源是否被分配，通常是記憶體中的 1 個 bit
 - clear (0)：資源可用
 - set (1)：資源已被分配，有行程正在用
- 每當有印表機存取請求時，作業系統檢查旗標
 - 若旗標為 **clear**，則允許請求並 **set 旗標**；
 - 若旗標已被 **set**，則讓行程 **wait**。
 - 每當行程使用完印表機後，作業系統再將印表機分配給下一個等待的行程，或是若無人等待，則 **clear 旗標**。
- 問題：測試與設定旗標可能需要多條指令才能完成，如果過程被中斷，就會搶資源

舉例來說：

假設印表機目前可用，而行程 A 請求使用印表機。

作業系統讀取旗標，發現它是清除的（代表印表機可用）。

但就在作業系統還沒來得及設置旗標時，行程 A 被中斷，行程 B 開始執行。

行程 B 也請求使用印表機。

作業系統再次讀取旗標，發現它仍是清除的（因為 A 還沒設置），

於是它也允許行程 B 使用印表機。

稍後，行程 A 恢復執行，繼續把旗標設置並開始列印。

結果：兩個行程同時使用同一台印表機。 ↓

- **互斥 Mutual Exclusion**：能讓行程安全共享資源的原則
- **Test-And-Set**：把讀取旗標到 set 旗標合在單一機器指令中

Deadlock

系統中多個行程彼此等待對方釋放資源，結果誰也無法繼續執行，整個系統陷入永久等待。

可能發生 Deadlock 的條件：

- 有資源不被共享，一次只能被一個行程佔用
- 有行程分多次取得所需資源，等待拿到資源再請求下一次
- 資源不可被強制奪走

例如 CPU 與 I/O 裝置在 Handshake 互等

I/O 裝置等 CPU 的 Acknowledge，CPU 等 I/O 的 Ready → 沒人做事

Deadlock 避免

- Deadlock Detection and Correction (死結偵測與修正)
 - 假設 Deadlock 不太可能發生，所以不主動避免。
 - 若真的 Deadlock，偵測後強制回收資源以解除死結。
 - 常見方法：
 - ◆ Kill process (終止行程)
 - ◆ 作業系統或系統管理員可「kill」一些行程，釋放資源。
 - 例：Process Table 滿時，系統可終止部分行程釋放空間。
 - 優點：簡單直接。
 - 缺點：可能導致重要任務中斷或資料遺失。
- Deadlock Avoidance (死結避免)
 - 透過設計讓死結**不會發生**。
 - ◆ 例：要求每個行程一次性請求所有需要的資源 (條件 2)
 - ◆ 例：將不可共享的資源轉變為可共享的資源 (條件 1)
→ Spooling

Spooling

概念：

- 讓多個行程看起來能同時使用一個輸出設備（例如印表機）。
- 實際上，作業系統將各行程的輸出先儲存在磁碟或大容量儲存裝置中（mass storage），等印表機空閒時再依序列印。

效果：

- 行程以為自己已取得印表機使用權。
- 實際上由系統排程統一輸出 → 模擬出可共享的資源。

應用延伸：

- 多個行程可同時讀取（read access）同一檔案。
- 但寫入（write access）通常只允許一個行程進行，以防資料衝突。
- 有些系統可將檔案分段，允許多個行程同時修改不同區域。

挑戰：

- 若某行程修改了檔案，必須決定：
 - ◆ 僅讀取的行程是否、以及如何被通知檔案變更。

Security

- 帳號：本質上是系統的一筆記錄，包含使用者名稱、密碼、其擁有的權限（**privileges**）
- 帳號管理：通常由 超級使用者（**super user**）或系統管理員（**administrator**）建立
- 外部攻擊：
 - 不安全的密碼：容易猜到、與他人共用、密碼外洩...
 - 惡意軟體
- 稽核軟體（**auditing software**）：分析系統中的活動協助系統管理員監控系統
 - 偵測大量錯誤的登入嘗試（可能是入侵者暴力破解密碼）
 - 發現帳號內不尋常的活動模式（例如平常只用文書軟體的使用者，突然執行技術性或管理性工具）
 - 偵測嗅探軟體（**sniffing software**）
- **sniffing software**
不一定是惡意軟體，也是一種正常的網路分析工具
「被動或主動監聽/捕捉網路上的封包或系統輸入」例如在網卡上抓取未加密的流量或鍵盤輸入的紀錄器（**keylogger**）
- **Spyware / Keylogger / Trojan / Fake-login**
偷帳號、資料的惡意軟體
 - * 課本把這類也寫在 **Sniffing Software**，課本應該是錯的，**Sniffing** 應該不會在目標主機上
在被害目標主機上執行，蒐集鍵盤輸入、螢幕、登入憑證或把使用者導向偽造畫面（類似於釣魚網站）以偷取帳號。

- 內部攻擊：

攻擊來源	內部攻擊來自己取得存取權的使用者或入侵者。
攻擊方式	嘗試突破作業系統權限限制，例如越界存取記憶體或未授權檔案。
CPU 防護機制	以「記憶體範圍暫存器」與「權限模式」限制程式操作範圍。
特權模式設計	● 特權模式 privileged mode 可執行所有機器指令 ● 非特權模式 nonprivileged mode 只能執行部分安全的指令
開機	開機時為特權模式 讓系統順利載入 開機後使用者程式執行時切換為非特權模式

潛在風險	權限控制錯誤會導致：time-slice 擅自被延長、直接讀寫設備、竊取他人資料等問題 可能導致資安問題或系統崩潰
------	--